



南開大學

Nankai University

计算机学院

自然语言处理实验报告

作业一：文本分类实现

姓名：李佳璞

学号：2314007

专业：计算机科学与技术

2026 年 9 月 28 日

目 录

1 实验目的	2
2 实验环境	2
3 数据集与实验设置	2
3.1 数据集	2
3.2 数据划分	2
3.3 评价指标	3
3.4 统一的训练与选模流程	3
4 实验过程与实现	4
4.1 Task 1: Bag of Words	4
4.2 Task 2: Word Embedding	4
4.3 Task 3: BERT Fine-tuning	5
4.4 运行方式	6
5 实验结果	6
5.1 总体结果	6
5.2 逐类别结果	7
5.3 验证集上的超参数选择	7
5.4 BERT 训练过程	8
6 分析与比较	8
6.1 整体表现与 Macro-F1 的作用	8
6.2 Binary Bag of Words 与 Word Frequency	8
6.3 词向量平均表示	9
6.4 BERT 的表现	9
6.5 方法对比小结	10
7 实验心得	10

1 实验目的

本次作业在同一个新闻分类任务上比较三类文本表示方法：传统词袋模型、词向量平均表示以及预训练语言模型微调。具体目标包括：

1. 理解 Binary Bag of Words 与 Word Frequency 两种词袋表示的区别；
2. 掌握文本分类的基本实验流程：数据划分、分词、特征构建、分类器训练与评价；
3. 使用 Logistic Regression 作为统一的分类器，比较不同文本表示带来的差异；
4. 掌握预训练 GloVe 词向量的使用，以及用 gensim 在 AG News 和 NYT 上自行训练 Word2Vec；
5. 熟悉 BERT 类预训练模型的 Fine-tuning 流程；
6. 使用 Accuracy 和 Macro-F1 评价模型，并对各方法的性能进行比较和分析。

2 实验环境

项目	说明
硬件	GPU: A40 , 显存 48GB, 驱动版本 550.54.14;
软件环境	Python 3.10, PyTorch 2.4.1+cu118, transformers 4.46.3, scikit-learn 1.5.2, gensim 4.3.3, nltk 3.9.1, numpy 1.26.4, pandas 2.2.3
预训练资源	GloVe 词向量文件 glove.6B.100d.txt, 包含 40 万个词, 每个词 100 维; 预训练模型 google-bert/bert-base-uncased
运行耗时	三个 Task 依次运行共 14 分 31 秒: Task 1 约 3 分钟, Task 2 约 8 分钟, Task 3 约 3 分钟, 其中 BERT 训练本身只用了 53 秒
代码仓库	https://github.com/c20201057/nlp/tree/HW1

表 1: 实验环境

3 数据集与实验设置

3.1 数据集

NYT 数据集保存在 `nyt.csv` 中, 共 11519 条新闻, 包含 `text` 和 `label` 两列, 类别为 `business`、`politics`、`sports` 三类。文本已经全部转为小写。按空格切分, 平均每篇约 638 个词, 中位数为 672 个词, 最短的一篇 33 个词, 最长的一篇 5134 个词。类别分布很不均衡: `sports` 占 75%, `business` 与 `politics` 各约 12.5%。数据中有 72 条完全重复的记录, 作业要求中没有去重的说明, 因此本实验保留原始数据。

AG News 数据集保存在 `ag.csv` 中, 共 90000 条新闻, 只有 `text` 一列, 平均每篇约 39 个词, 仅用于训练 Word2Vec 词向量。

3.2 数据划分

按作业要求, 以随机种子 42 对 NYT 进行随机打乱, 再按 80%、10%、10% 的比例划分为训练集、验证集和测试集。所有实验都调用同一个划分函数 `load_nyt_splits()`, 划分下标保存在 `results/nyt_split.json` 中, 保证不同方法之间可以直接比较。各集合的类别分布如表 2 所示。

集合	business	politics	sports	合计
Training Set	1157	1134	6924	9215
Validation Set	124	152	875	1151
Test Set	148	165	840	1153

表 2: NYT 数据划分与类别分布

3.3 评价指标

设测试集共 N 个样本，类别集合为 $\mathcal{C}$ 。Accuracy 为分类正确的样本占比：

$$\text{Accuracy} = \frac{1}{N} \sum_{i=1}^N 1[\hat{y}_i = y_i]$$

Macro-F1 先对每个类别 c 分别计算精确率 P_c 、召回率 R_c 与 $F1_c = \frac{2P_cR_c}{P_c+R_c}$ ，再对所有类别取算术平均：

$$\text{Macro-F1} = \frac{1}{|\mathcal{C}|} \sum_{c \in \mathcal{C}} F1_c$$

由于 sports 占测试集的 72.9%，一个把所有样本都判为 sports 的平凡分类器也能得到 0.7285 的 Accuracy，但它的 Macro-F1 只有 0.281。因此 Macro-F1 更能反映模型在 business、politics 这两个少数类上的表现。两个指标均使用 scikit-learn 的 `accuracy_score` 与 `f1_score` 计算，后者设置 `average="macro"`。

3.4 统一的训练与选模流程

Task 1 与 Task 2 统一使用 scikit-learn 的 `LogisticRegression`，求解器为 `lbfgs`，`max_iter` 设为 5000。正则强度 C 在 0.01、0.1、1、10、100 中选择：每个 C 在训练集上训练，在验证集上计算 Macro-F1，取验证集 Macro-F1 最高的模型在测试集上报告结果。测试集只在最后评测时使用一次，不参与任何选择。

代码 1: 节选自 `common.py`: 按验证集选择 C 的 LR 训练函数

```

1 for C in C_grid:
2     clf = LogisticRegression(C=C, max_iter=5000, random_state=SEED)
3     if scaler:
4         clf = make_pipeline(StandardScaler(), clf)
5     clf.fit(X_train, y_train)
6     val = evaluate(y_val, clf.predict(X_val), labels)
7     if best is None or val["macro_f1"] > best[1]["macro_f1"]:
8         best = (clf, val, C)
9 clf, val, C = best
10 test = evaluate(y_test, clf.predict(X_test), labels)

```

分词统一使用 `nltk.word_tokenize`。分词结果缓存到 `cache/` 目录，Task 1 与 Task 2 共用同一份分词结果。

4 实验过程与实现

4.1 Task 1: Bag of Words

词表只由训练集构建。训练集分词后共有 $|V| = 128343$ 个不同的词, 每篇文档表示为一个 $|V|$ 维向量。由于绝大多数维度为 0, 向量以稀疏矩阵存储。若使用稠密矩阵, 仅训练集就需要约 $9215 \times 128343 \times 8$ 字节, 也就是约 9.5GB 内存。

两种表示的区别只在于是否记录词频:

- **Binary Bag of Words**: 若词表中第 i 个词出现在文档中则 $x_i = 1$, 否则 $x_i = 0$;
- **Word Frequency**: $x = (c_1, c_2, \dots, c_{|V|})$, 其中 c_i 为第 i 个词在文档中出现的次数。

实现上使用 `CountVectorizer`, 并传入已经分好词的 token 列表, 由 `binary` 参数切换两种表示。验证集与测试集中不在训练词表内的词被忽略。

代码 2: 节选自 `task1_bow.py`: 两种词袋表示的构建

```
1 for name, binary in [("task1_binary_bow", True), ("task1_word_frequency", False)]:
2     vec = CountVectorizer(analyzer=identity, binary=binary)
3     X_train = vec.fit_transform(tokens["train"])
4     X_val, X_test = vec.transform(tokens["val"]), vec.transform(tokens["test"])
5     result = fit_lr_with_val(X_train, y["train"], X_val, y["val"],
6                             X_test, y["test"], labels)
```

4.2 Task 2: Word Embedding

三组实验都使用 100 维词向量, 文档向量由文档中所有有效单词的词向量取平均得到:

$$e(d) = \frac{1}{n} \sum_{i=1}^n e(w_i)$$

其中“有效单词”指在该组词向量词表中出现过的词, 未登录词直接跳过。三组实验中每篇文档都至少有一个有效单词, 因此没有出现零向量文档。得到的稠密文档向量先经过 `StandardScaler` 标准化, 均值和方差只在训练集上估计, 然后再送入 Logistic Regression。

代码 3: 节选自 `task2_embeddings.py`: 文档向量计算

```
1 def doc_vectors(docs, lookup):
2     X = np.zeros((len(docs), DIM), dtype=np.float32)
3     for i, doc in enumerate(docs):
4         vecs = [lookup[w] for w in doc if w in lookup]
5         if vecs:
6             X[i] = np.mean(vecs, axis=0)
7     return X
```

Pre-trained GloVe. 读取 `glove.6B.100d.txt` 中的 40 万个词向量。GloVe 6B 在约 60 亿词的 Wikipedia 2014 与 Gigaword 5 语料上训练, 其中 Gigaword 本身就是新闻语料。NYT 测试集的 token 覆盖率为 98.06%。

Word2Vec Trained on AG News. 将 AG News 的 90000 篇文本分词后, 用 `gensim` 训练 100 维 Word2Vec, 训练语料约 350 万词。训练参数如表 3 所示, 得到 43870 个词的词表。NYT 测试集的 token 覆盖率为 95.51%, 是三组中最低的。

Word2Vec Trained on NYT. 使用同样的参数在 NYT 上训练，得到 57775 个词的词表，测试集 token 覆盖率为 98.37%。为了避免验证集和测试集文本参与表示学习，这里只使用 NYT 训练集的 9215 篇文本训练词向量，训练语料约 590 万词。

参数	取值
模型	Skip-gram, <code>sg=1</code> ; 负采样, <code>negative=5</code>
向量维度	<code>vector_size=100</code>
上下文窗口	<code>window=5</code>
最低词频	<code>min_count=2</code>
训练轮数	<code>epochs=10</code>
随机性	<code>seed=42</code> , <code>workers=1</code> , 单线程训练保证结果可复现

表 3: Word2Vec 训练参数，AG News 与 NYT 两组相同

AG News 语料中的新闻较短，把每篇新闻作为一个“句子”输入即可。NYT 单篇新闻最长约 5000 词，没有超过 gensim 每个句子 10000 词的上限，因此同样按篇输入。为了确认结果可以复现，全部实验先后在 A40 上完整运行了两次，两次得到的所有指标完全相同。

4.3 Task 3: BERT Fine-tuning

使用 `google-bert/bert-base-uncased`，在 [CLS] 表示之上接一个随机初始化的 3 类线性分类头，即 `BertForSequenceClassification`，对整个模型进行微调。按作业要求，tokenization 时设置 `max_length=64`，超出部分截断，不足部分补齐，共训练 3 个 epoch。其余超参数如表 4 所示。

参数	取值
预训练模型	<code>bert-base-uncased</code> ，约 1.1 亿参数
最大序列长度	64
训练轮数	3
Batch size	32，每个 epoch 288 步，共 864 步
优化器	AdamW，学习率 2×10^{-5} ，weight decay 0.01，bias 与 LayerNorm 参数不做衰减
学习率调度	前 10% 的步数线性 warmup，之后线性衰减到 0
其他	梯度裁剪阈值 1.0；bf16 混合精度；随机种子 42
模型选择	报告第 3 个 epoch 结束时的模型，不根据验证集提前停止

表 4: BERT 微调设置

代码 4: 节选自 `task3_bert.py`: 训练循环

```

1 for epoch in range(1, args.epochs + 1):
2     model.train()
3     for input_ids, attention_mask, y in train_loader:
4         with torch.autocast("cuda", dtype=torch.bfloat16):

```

```

5         loss = model(input_ids=input_ids.to(device),
6                     attention_mask=attention_mask.to(device),
7                     labels=y.to(device)).loss
8     loss.backward()
9     torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
10    optimizer.step()
11    scheduler.step()
12    optimizer.zero_grad()
13    val = evaluate(y_val, predict(model, val_loader, device), labels)

```

4.4 运行方式

三个 Task 由脚本 `scripts/run_all.sh` 依次执行，结果以 JSON 形式写入 `results/`，完整输出写入 `logs/run_all.log`。BERT 权重事先下载到 `models/bert-base-uncased`，运行时以离线模式从本地加载。

代码 5: 节选自 `scripts/run_all.sh`

```

1 export PYTHONNOUSERSITE=1
2 export HF_HUB_OFFLINE=1 TOKENIZERS_PARALLELISM=false
3 exec > >(tee logs/run_all.log) 2>&1
4 "$PY" -u src/task1_bow.py
5 "$PY" -u src/task2_embeddings.py
6 "$PY" -u src/task3_bert.py --model models/bert-base-uncased

```

5 实验结果

5.1 总体结果

六组实验在 NYT Test Set 上的结果如表 5 所示，测试集共 1153 条。表中的“错分数”由 Accuracy 换算得到。

Task	方法	选定 C	Accuracy	Macro-F1	错分数
Task 1	Binary Bag of Words + LR	0.1	0.9870	0.9712	15
	Word Frequency + LR	100	0.9879	0.9724	14
	GloVe 6B-100d + LR	0.1	0.9922	0.9826	9
Task 2	AG News 训练的 Word2Vec + LR	0.1	0.9844	0.9653	18
	NYT 训练的 Word2Vec + LR	0.01	0.9879	0.9725	14
Task 3	BERT-base-uncased 微调	—	0.9792	0.9530	24

表 5: NYT Test Set 上的 Accuracy 与 Macro-F1

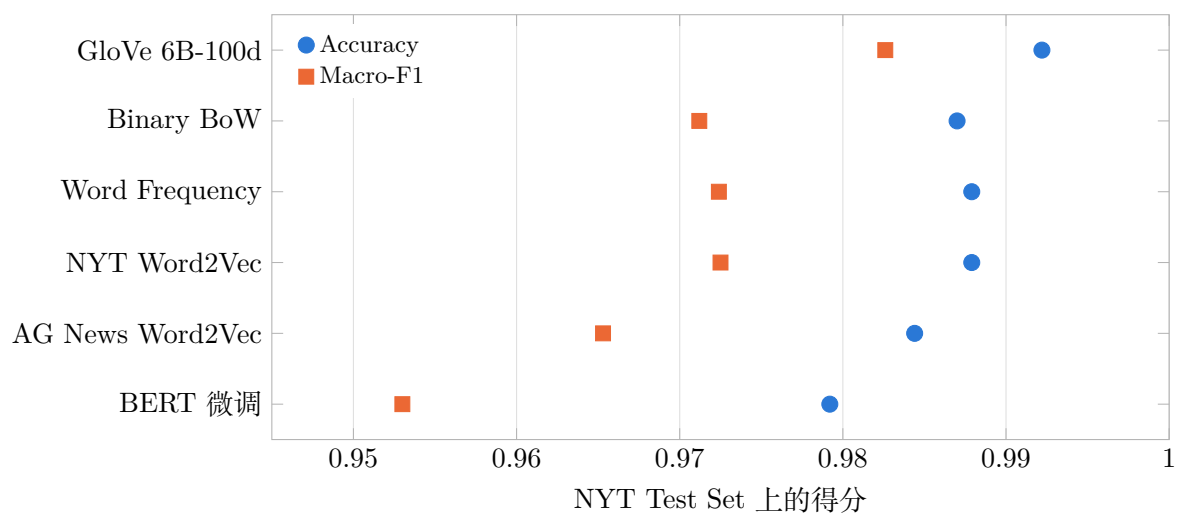


图 5.1: 六种方法的 Accuracy 与 Macro-F1，按 Macro-F1 从低到高排列，横轴从 0.945 开始以便显示差异

5.2 逐类别结果

表 6 给出测试集上每个类别的 F1。所有方法在 sports 上的 F1 都接近 1，各方法 Macro-F1 的差异几乎全部来自 business 和 politics 两个少数类。

方法	business F1	politics F1	sports F1
Binary Bag of Words	0.9550	0.9610	0.9976
Word Frequency	0.9550	0.9641	0.9982
GloVe 6B-100d	0.9763	0.9726	0.9988
AG News 训练的 Word2Vec	0.9521	0.9461	0.9976
NYT 训练的 Word2Vec	0.9553	0.9639	0.9982
BERT 微调	0.9324	0.9297	0.9970

表 6: 测试集逐类别 F1

5.3 验证集上的超参数选择

表 7 列出每个 C 在验证集上的 Macro-F1，加粗的是最终选中的 C 。

方法	$C = 0.01$	$C = 0.1$	$C = 1$	$C = 10$	$C = 100$
Binary Bag of Words	0.9661	0.9760	0.9711	0.9745	0.9720
Word Frequency	0.9619	0.9611	0.9627	0.9658	0.9756
GloVe 6B-100d	0.9545	0.9597	0.9568	0.9497	0.9438
AG News 训练的 Word2Vec	0.9581	0.9625	0.9586	0.9598	0.9514
NYT 训练的 Word2Vec	0.9677	0.9645	0.9556	0.9467	0.9395

表 7: 不同正则强度下的验证集 Macro-F1

5.4 BERT 训练过程

Epoch	训练损失	验证集 Accuracy	验证集 Macro-F1	耗时
1	0.2814	0.9705	0.9277	24.1 s
2	0.0496	0.9713	0.9297	14.2 s
3	0.0182	0.9739	0.9369	14.4 s

表 8: BERT 每个 epoch 的训练损失与验证集结果

第 1 个 epoch 耗时较长，可能是因为包含了 CUDA 与计算核的初始化开销。训练损失从 0.28 降到 0.018，而验证集 Macro-F1 只从 0.928 升到 0.937，模型对训练集已经拟合得很好，但泛化到验证集的提升有限。

6 分析与比较

6.1 整体表现与 Macro-F1 的作用

六种方法的 Accuracy 都在 0.979 以上，最好与最差之间只相差 15 个测试样本。原因是这三类新闻的主题差异明显，尤其 sports 类有大量球队、比分、赛事之类的专属词汇，因此即使是最简单的词袋模型也能达到很高的准确率。这时 Accuracy 主要由占 72.9% 的 sports 决定，不容易区分方法好坏；Macro-F1 给三个类别相同的权重，更能体现各方法在 business 和 politics 上的区别。例如 BERT 与 Binary BoW 的 Accuracy 只差 0.8 个百分点，Macro-F1 却差了 1.8 个百分点。

需要注意的是，business 和 politics 在测试集中只有 148 条和 165 条，单个样本的错分就会让对应类别的 F1 变化约 0.3 个百分点。因此 Word Frequency 与 NYT 训练的 Word2Vec 之间 0.0001 的 Macro-F1 差距，以及 Binary BoW 与 Word Frequency 之间 1 个样本的差距，都不应理解为方法本身的优劣，只能认为两者表现相当。

6.2 Binary Bag of Words 与 Word Frequency

两种词袋表示的测试结果几乎相同，分别错分 15 条与 14 条。NYT 新闻平均长度约 640 词，一个词“是否出现”已经携带了判别主题所需的大部分信息。词频虽然额外记录了重复次数，但高频词多为功能词，对主题分类的帮助有限。

两者的差别主要体现在所选的正则强度上：Binary BoW 在 $C = 0.1$ 时最好，而 Word Frequency 在网格上限 $C = 100$ 时最好。词频特征的取值范围比 0/1 特征大得多，长文档中的高频词会产生很大的特征值，同样的 C 对它们的约束效果也不同。Word Frequency 选中的 C 落在网格边界，说明更大的 C 可能还有提升空间；本实验按统一网格进行，没有针对它单独扩展搜索范围。

6.3 词向量平均表示

GloVe 是六种方法中最好的，测试集只错分 9 条，Macro-F1 达到 0.9826，business 和 politics 的 F1 均明显高于其他方法。可能的原因有两点。其一，GloVe 在约 60 亿词的 Wikipedia 与 Gigaword 新闻语料上训练，语料规模比 AG News 和 NYT 训练集大三个数量级，词义表示更充分，且与 NYT 同属新闻领域。其二，100 维的平均向量把语义相近的词映射到相近的位置，相比 12.8 万维的稀疏词袋，分类器需要学习的参数少得多，在 business 和 politics 各只有约 1150 条训练样本的情况下更不容易过拟合。

NYT 训练的 Word2Vec 优于 AG News 训练的 Word2Vec，错分数分别为 14 和 18。两者训练方法和参数完全相同，差异来自训练语料：

- **领域匹配**：NYT 词向量在分类任务本身的语料上训练，词的共现关系与分类时的用法一致；AG News 虽然也是新闻，但以短讯为主，来源与写作风格和 NYT 不同；
- **词表覆盖**：AG News 词向量对 NYT 测试集的 token 覆盖率只有 95.5%，而 NYT 词向量为 98.4%，前者有更多词在取平均时被丢弃；
- **语料规模**：AG News 约 350 万词，NYT 训练集约 590 万词，后者更大。

两组自训练词向量都不如 GloVe，说明在这个规模下，预训练语料的规模和质量比“与任务同领域”更重要。NYT 训练的 Word2Vec 与两种词袋表示的结果相当，说明在同一份训练数据上，100 维的平均词向量基本保留了词袋表示中对分类有用的信息。

词向量平均方法也有明显的局限：取平均会丢失词序，也会把大量与主题无关的词一起平均进来。它能在本任务上表现很好，是因为主题分类主要依赖“文中出现了哪些领域词”，对词序并不敏感。

6.4 BERT 的表现

BERT 是参数最多、预训练最充分的模型，但在本实验中得分最低，Macro-F1 为 0.9530，错分 24 条，差距主要来自 business 与 politics 两类，F1 分别只有 0.9324 和 0.9297。我认为主要原因是 `max_length=64` 带来的截断：

- NYT 新闻平均约 640 词，而 64 个 WordPiece token 扣除 [CLS] 与 [SEP] 后只能容纳约 50 个词，模型实际只看到每篇文章开头不到 10% 的内容，其余方法使用的都是全文；
- business 与 politics 新闻都常涉及政府、公司、政策等内容，只看开头几十个词时两者可能难以区分，区分它们的关键信息也可能出现在后文；sports 新闻开头的球队、比分等特征非常明显，所以 sports 的 F1 仍然接近 1；
- 表 8 中训练损失降到 0.018，验证集 Macro-F1 却只有 0.937，模型已经能很好地拟合训练集，这与“输入信息不足”的解释相符。

此外，3 个 epoch 内验证集 Macro-F1 仍在上升，训练轮数也可能限制了 BERT 的表现，但作业规定为 3 个 epoch，本实验没有调整。需要说明的是，上述截断解释是根据文本长度与逐类别结果作出的推断，本实验没有做对照实验来直接验证。可行的对照是：让词袋或 GloVe 方法同样只使用每篇文章的前 64 个 token，或者把 BERT 的 `max_length` 提高到 512，观察差距是否随之缩小。

6.5 方法对比小结

方法	优点	局限
词袋模型	实现简单、可解释，使用全文信息，在主题分类上已经很强	维度高达 12.8 万且非常稀疏，无法利用近义词之间的关系，未登录词直接丢弃
词向量平均	维度低，能利用词义相似性；大规模预训练的 GloVe 效果最好	丢失词序；效果依赖词向量的训练语料与覆盖率，小语料自训练的词向量提升有限
BERT 微调	上下文相关的表示能力最强，1 分钟内即可完成微调	受最大序列长度限制，长文档只能看到开头；计算开销远高于前两类方法

表 9: 三类文本表示方法的比较

7 实验心得

本次作业最大的体会是，表示能力更强的模型不一定在每个任务上都表现更好，要结合任务和实验设置来分析。在 NYT 三分类这样主题差异明显的任务上，全文词袋和 GloVe 平均向量都已经非常强，而 BERT 受 64 个 token 的长度限制，只能看到文章开头，反而表现最差。如果只看模型规模就预期 BERT 最好，就会误判结果。

第二点体会是评价指标的选择。类别不均衡时，Accuracy 很容易被多数类“撑高”，必须结合 Macro-F1 和逐类别 F1 才能看清模型在少数类上的问题。同时测试集中少数类样本不多，几个样本的差异不足以说明方法的优劣，比较结果时要考虑这种随机波动。

第三点是实验流程的规范性。所有方法共用同一份数据划分，超参数只在验证集上选择，测试集只在最后使用一次；NYT 的 Word2Vec 也只用训练集训练，避免测试集信息泄漏到表示学习中。固定随机种子、单线程训练 Word2Vec、记录运行日志，都是为了让结果能够被复现和检查。全部实验完整运行两次得到了完全相同的结果，也验证了这一点。

最后，把环境搭建、资源下载和实验运行整理成脚本，并把依赖版本固定在 requirements.txt 中，也让整个实验可以在新的机器上重复运行。